

ShowPilot — SMPTE-Triggered Song Lyrics

Feature spec for Claude Code execution Priority: Critical — requested by ~90% of founding church testers **Author:** Rey Disse / CTO session **Date:** September 2026

What This Is

Timecode-driven song lyric automation. A worship band runs Ableton Live (or any DAW) with backing tracks. Ableton sends MIDI Timecode (MTC) to ShowPilot. ShowPilot receives the timecode and automatically fires lyric slide transitions at pre-mapped cue points — no operator clicking required.

Two operating modes, following ShowPilot's "integration first, native fallback" philosophy:

1. **ProPresenter integration** — timecode triggers advance slides in a connected ProPresenter instance via the Bridge
2. **Native lyrics engine** — ShowPilot renders lyrics on its own kiosk/stage display, no ProPresenter needed

Both modes consume the same cue map data structure. The operator picks their target in settings.

How Ableton Fits In

Ableton Live is the most common DAW in church worship. The band runs:

- **Click track** (headphone feed for the drummer/band)
- **Backing tracks / stems** (pads, loops, strings, keys)
- **MIDI Timecode (MTC)** — Ableton sends MTC out of a MIDI port

Signal chain — Tauri desktop app + Bridge

ShowPilot runs as a **Tauri desktop app**, and the Bridge is a separate **Tauri app** running on the booth machine (or the same machine). MIDI input goes through the Bridge, not the browser.

Ableton Live → MTC out → USB MIDI interface → Bridge (Tauri / Rust midir) → WebSo

The Bridge Tauri app reads MTC natively via Rust's `midir` crate — no browser permissions, no Chrome dependency, no Web MIDI API flakiness. The Bridge decodes incoming MTC quarter-frame messages into full SMPTE timecode values, then sends `feed-tc` commands to TimecodeRelay over its existing WebSocket connection to ShowPilot cloud. This is the same `feed-tc` command the browser-based `MtcSource` sends today — TimecodeRelay doesn't care who sends it.

What the Bridge needs (new):

- A Rust-side MIDI input module using `midir` that enumerates available MIDI inputs and decodes MTC quarter-frame messages into `TimecodeValue` structs
- A configuration UI (in the Bridge Tauri app) to select which MIDI input to listen on
- The Bridge's WebSocket connection to ShowPilot already exists — it just needs to send `feed-tc` messages with the decoded timecode at ~10Hz, same cadence as the browser `MtcSource`

What the Bridge already does:

- Connects to ShowPilot cloud via authenticated WebSocket
- Sends and receives commands
- Manages ProPresenter connections on the local network

Fallback: The existing browser-based Web MIDI path (`MtcSource` in `src/lib/timecode-sources/mtc-source.ts`) remains available for users who access ShowPilot via a web browser on Chrome/Edge. The desktop app should prefer the Bridge path. Both produce identical `feed-tc` messages — TimecodeRelay is source-agnostic.

Ableton setup (user-facing docs):

1. In Ableton Preferences → MIDI, enable "Sync" output on the MIDI port connected to the booth's USB interface
2. In the Bridge app, select that MIDI input from the dropdown
3. Hit play in Ableton — timecode flows automatically

The feature itself is about **what happens when the cue fires**, not the timecode transport — the transport is solved.

Architecture — What Exists, What's New

Already built (no changes needed)

- `TimecodeRelay` Durable Object with automation event engine (`evaluateEvents` , `crossedTriggerFrame`)
- `useTimecode` hook with event management (`addEvent` , `updateEvent` , `removeEvent`)
- Browser MTC source via Web MIDI API (fallback path — see Bridge MTC below)
- Internal freerun generator (for rehearsal/testing without Ableton)
- ProPresenter Bridge (Tauri) with `sendCommand()` support (`next` , `previous` , `clear`)
- Bridge WebSocket connection to ShowPilot cloud
- Kiosk/stage display infrastructure
- Automation timeline UI with event editor

Already built (needs extension)

- **Bridge Tauri app** — needs MTC input module (Rust `midir` crate) to receive MIDI timecode and send `feed-tc` to `TimecodeRelay`. Currently handles ProPresenter + ATEM + device protocols but not MIDI input.
- **ProPresenter Bridge protocol** — currently reads slides FROM PP and sends `next` / `previous` / `clear` commands. Needs: `trigger-slide` (specific slide by index), `list-presentations` (enumerate all loaded presentations with slide content for lyrics import).

New — add to existing

1. New automation action types (types + relay + editor)

Add to `AutomationActionType` in `src/types/timecode.ts`:

```
| "lyrics-goto"           // Jump to a specific lyric slide
| "lyrics-clear"         // Clear lyrics display
| "pp-trigger-slide"     // Trigger a specific ProPresenter slide by index
| "pp-trigger-next"      // Advance ProPresenter to next slide
| "pp-trigger-clear"     // Clear ProPresenter slide layer
```

Add to `SUPPORTED_ACTIONS` in `TimecodeRelay.ts`.

Add execution handlers in `executeEventAction`:

- `lyrics-goto` → broadcast a `lyrics-update` message to all connected kiosk/stage clients (new WS message type on `ChatRelay` or a dedicated `LyricsRelay` — see decision below)
- `lyrics-clear` → broadcast lyrics clear
- `pp-trigger-slide` → send command to Bridge via `BridgeRelay` (new command: `pp-trigger-slide` with `{ presentationIndex, slideIndex }`)
- `pp-trigger-next` / `pp-trigger-clear` → send `next` / `clear` to Bridge (already supported)

2. Bridge Tauri — MTC input module (Rust)

New Rust module in the Bridge Tauri app for receiving MIDI Timecode:

Crate: `midir` (cross-platform MIDI I/O for Rust)

Responsibilities:

- Enumerate available MIDI input ports and expose them to the Tauri frontend via a `list_midi_inputs` command
- Open a selected MIDI input and decode MTC quarter-frame messages (0xF1 status byte) into full SMPTE timecode values
- MTC sends 2 quarter-frame messages per frame across 8 messages (4 frames to build a full TC value). The decoder must accumulate quarter-frames and emit a complete `TimecodeValue` every 4 frames
- Auto-detect frame rate from the MTC type bits (24, 25, 29.97df, 30fps)
- Feed decoded timecode to the Bridge's WebSocket connection as `feed-tc` commands at ~10Hz (same cadence as the browser `MtcSource`)
- Handle MIDI device hot-plug: re-enumerate on connect/disconnect, auto-reconnect to the previously selected input if it reappears

Tauri commands:

```
#[tauri::command]
fn list_midi_inputs() -> Vec<MidiInputInfo> { ... }

#[tauri::command]
fn start_mtc_input(port_name: String) -> Result<(), String> { ... }
```

```
#[tauri::command]
fn stop_mtc_input() -> Result<(), String> { ... }
```

Bridge config UI: Add a "MIDI Timecode" section to the Bridge Tauri app's settings panel — dropdown of available MIDI inputs, Start/Stop button, status indicator showing decoded TC.

3. Expanded ProPresenter Bridge commands

In the ProPresenter bridge protocol, expand command support:

Trigger specific slides:

```
PP7 API endpoints:
GET /v1/presentation/active/{slideIndex}/trigger    - trigger slide by index i
GET /v1/presentation/{uuid}/{slideIndex}/trigger    - trigger slide by index
GET /v1/playlist/active/{index}/trigger              - trigger playlist item b
```

Update `sendCommand` to accept a payload object (not just a string command name) so slide index and presentation UUID can be passed through.

List presentations (new — for lyrics import):

```
PP7 API endpoints:
GET /v1/presentations                               - list all presentations
GET /v1/presentation/{uuid}                         - get full presentation w
GET /v1/playlists                                    - list all playlists
GET /v1/playlist/{uuid}                             - get playlist items
```

Add a `query-presentations` command to the Bridge protocol. When the ShowPilot frontend requests it (via BridgeRelay), the Bridge queries PP's local API, collects all presentations with their slide text, and returns the data over WebSocket. This is a query/response, not a continuous feed — it runs once when the operator opens the "Import from ProPresenter" flow.

Response shape:

```
interface PPPresentation {
  uuid: string;
  name: string;           // e.g., "Amazing Grace"
  slides: PPSlide[];
}

interface PPSlide {
```

```

    index: number;
    text: string;           // The lyric text on this slide
    label: string;          // PP's group/arrangement label if present (e.g., "Ver
    notes: string;          // Slide notes
}

```

4. Song data model (D1 schema)

New migration NNNN_songs.sql :

```

CREATE TABLE songs (
    id            TEXT PRIMARY KEY,
    orgId         TEXT NOT NULL,
    title         TEXT NOT NULL,
    artist        TEXT DEFAULT '',
    ccliNumber    TEXT DEFAULT '',
    bpm           INTEGER,
    keySignature  TEXT DEFAULT '',
    ppPresentationUuid TEXT DEFAULT '', -- ProPresenter presentation UUID if impor
    importSource  TEXT DEFAULT 'manual', -- 'manual' | 'propresenter'
    createdAt     TEXT NOT NULL DEFAULT (datetime('now')),
    updatedAt     TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (orgId) REFERENCES organization(id)
);

```

```

CREATE INDEX idx_songs_org ON songs(orgId);

```

```

CREATE TABLE song_sections (
    id            TEXT PRIMARY KEY,
    songId        TEXT NOT NULL,
    orgId         TEXT NOT NULL,
    label         TEXT NOT NULL,          -- "Verse 1", "Chorus", "Bridge", etc.
    lyrics        TEXT NOT NULL DEFAULT '',
    sortOrder     INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (songId) REFERENCES songs(id) ON DELETE CASCADE,
    FOREIGN KEY (orgId) REFERENCES organization(id)
);

```

```

CREATE INDEX idx_song_sections_song ON song_sections(songId);

```

```

CREATE TABLE song_cue_maps (
    id            TEXT PRIMARY KEY,
    songId        TEXT NOT NULL,
    orgId         TEXT NOT NULL,
    name          TEXT NOT NULL DEFAULT 'Default', -- e.g., "Sunday AM arrangement", "Y
    format        TEXT NOT NULL DEFAULT '{"frameRate":30,"dropFrame":"ndf"}',

```

```

    createdAt TEXT NOT NULL DEFAULT (datetime('now')),
    updatedAt TEXT NOT NULL DEFAULT (datetime('now')),
    FOREIGN KEY (songId) REFERENCES songs(id) ON DELETE CASCADE,
    FOREIGN KEY (orgId) REFERENCES organization(id)
);

CREATE INDEX idx_song_cue_maps_song ON song_cue_maps(songId);

CREATE TABLE song_cues (
  id          TEXT PRIMARY KEY,
  cueMapId    TEXT NOT NULL,
  orgId       TEXT NOT NULL,
  sectionId   TEXT NOT NULL,      -- which song_section to show
  triggerTc   TEXT NOT NULL,      -- JSON TimecodeValue: {"hours":0,"minutes":1,
  triggerFrame INTEGER NOT NULL,
  sortOrder   INTEGER NOT NULL DEFAULT 0,
  FOREIGN KEY (cueMapId) REFERENCES song_cue_maps(id) ON DELETE CASCADE,
  FOREIGN KEY (sectionId) REFERENCES song_sections(id),
  FOREIGN KEY (orgId) REFERENCES organization(id)
);

CREATE INDEX idx_song_cues_map ON song_cues(cueMapId);

```

Design notes:

- `song_cue_maps` supports multiple arrangements per song (the youth band may have a different arrangement than Sunday AM — different section order, different timecodes)
- `song_sections` are the reusable lyric blocks; `song_cues` map them to timecodes within a specific arrangement
- All tables have `orgId` for tenancy scoping — **non-negotiable**
- CCLI number field for future SongSelect import
- `ppPresentationUuid` on songs tracks the source PP presentation — when the operator loads cues for a PP-imported song, ShowPilot already knows the correct slide indices (no manual mapping needed)
- `importSource` tracks provenance for UI hints ("imported from ProPresenter" badge on the song card)

5. Song CRUD server functions

New file: `src/lib/songs.ts`

Standard ShowPilot pattern: `createServerFn` with `inputValidator` + `handler`.

Auth/permission assertion first line. Functions:

- `getSongs(orgId)` — list all songs for org
- `getSong(songId)` — get song with sections and cue maps
- `createSong({ orgId, title, artist?, ccliNumber?, bpm?, keySignature? })`
- `updateSong(songId, updates)`
- `deleteSong(songId)` — cascades to sections and cue maps
- `createSongSection({ songId, label, lyrics, sortOrder })`
- `updateSongSection(sectionId, updates)`
- `deleteSongSection(sectionId)`
- `reorderSongSections(songId, orderedIds[])`
- `createSongCueMap({ songId, name, format })`
- `deleteSongCueMap(cueMapId)`
- `setSongCues(cueMapId, cues[])` — bulk replace all cues for a map (simpler than individual CRUD for the tap-to-mark workflow)

Permission: `songs:manage` for write ops, `songs:access` for read. Add to `ADMIN_TIER_PERMISSIONS` so TD/CD/PD all get it.

6. ProPresenter lyrics import

The highest-value import path. Most churches already have their songs built in ProPresenter — importing from PP means zero manual lyric entry.

Flow:

1. Operator clicks **"Import from ProPresenter"** on the Song Library page (or "Add Lyrics From → ProPresenter" in the Song Editor)
2. ShowPilot sends a `query-presentations` command to the Bridge via BridgeRelay
3. Bridge queries PP's local API (`GET /v1/presentations`), then fetches full slide content for each (`GET /v1/presentation/{uuid}`)
4. Bridge returns the list of `PPPresentation` objects over WebSocket

5. ShowPilot displays a picker: list of PP presentations with slide count previews
6. Operator selects one (or multiple for bulk import)
7. For each selected presentation, ShowPilot creates:
 - A `songs` row with `title` from PP presentation name, `ppPresentationUuid` stored, `importSource: 'propresenter'`
 - A `song_sections` row per slide — `label` from PP's group label (if present, otherwise auto-generated: "Slide 1", "Slide 2"), `lyrics` from slide text
8. Song appears in the library, ready for cue mapping

Key advantage: When the operator later loads cues targeting ProPresenter (not native lyrics), ShowPilot already knows the `ppPresentationUuid` and the slide indices match 1:1 with the imported sections. No manual mapping needed — the cue map auto-generates `pp-trigger-slide` events with the correct indices because the sections were imported from that same presentation.

Re-import / sync: If the operator re-imports a presentation that was previously imported (matched by `ppPresentationUuid`), ShowPilot shows a diff: "3 slides changed, 1 added." The operator can merge or replace. This handles the common case of a worship leader updating lyrics in PP during the week.

UI: Import picker

Import from ProPresenter

Connected to ProPresenter 7 at 192.168.1.50

Found 47 presentations

 Search presentations...

☐ Amazing Grace

(8 slides)

☐ Great Are You Lord

(6 slides)

☐ How Great Is Our God

(7 slides)

☐ Living Hope

(9 slides)

☐ O Come to the Altar

(6 slides)

...

Import 0 Selected

```
|  ⓘ Already imported: 12 songs (will skip or  
|    offer to sync if selected) |  
|_____|
```

Requires: Bridge connected with ProPresenter configured and reachable.

Fallback when PP not available: manual paste entry, always available.

7. Lyrics display renderer

New component: `src/components/lyrics/LyricsDisplay.tsx`

This renders on:

- The **kiosk/stage display** (full-screen, confidence-monitor style)
- An **embedded preview** in the song cue editor

The renderer receives:

- Current section label + lyrics text
- Next section label (for a "up next" preview line)
- Song title
- Optional: current timecode overlay

Styling: large, high-contrast text on dark background. Font size auto-scales to fill the viewport. Worship lyric standard: center-aligned, max 4 lines visible, clean sans-serif. Must look good on a 1080p confidence monitor and on a phone-sized kiosk preview.

The kiosk route (`/kiosk/$orgSlug` or similar) subscribes to a lyrics WebSocket channel.

When a `lyrics-goto` automation event fires, TimecodeRelay broadcasts the active section data. The kiosk renders it.

Decision: dedicated LyricsRelay DO vs. piggybacking on existing infra.

Recommendation: **Do NOT create a new Durable Object.** Instead, have TimecodeRelay broadcast a `lyrics-update` WebSocket message type when a `lyrics-goto` event fires. The kiosk client subscribes to the timecode WebSocket (it may already for the TC overlay) and listens for `lyrics-update` messages. This keeps the architecture simple — one fewer DO, one fewer WebSocket connection per client, and the lyrics state is inherently coupled to the timecode state anyway.

Add to `TimecodeWsMessage` :

```
| { type: "lyrics-update"; section: { label: string; lyrics: string } | null; son
```

8. "Load Song Cues" — connecting songs to the automation timeline

The cue map is the bridge between the song data and the automation engine. When an operator loads a song's cue map into the current timecode session:

1. Call `setSongCues data` → convert each cue into an `AutomationEvent` with action `lyrics-goto` and payload `{ sectionId, label, lyrics, songTitle }`
2. Bulk `addEvent` to `TimecodeRelay`
3. Optionally also generate `pp-trigger-slide` events if `ProPresenter` mode is active (mapping section order to PP slide indices)

This is a client-side operation: the Song Cue Editor page calls `addEvent` for each cue. The `TimecodeRelay` doesn't need to know about songs — it just fires automation events.

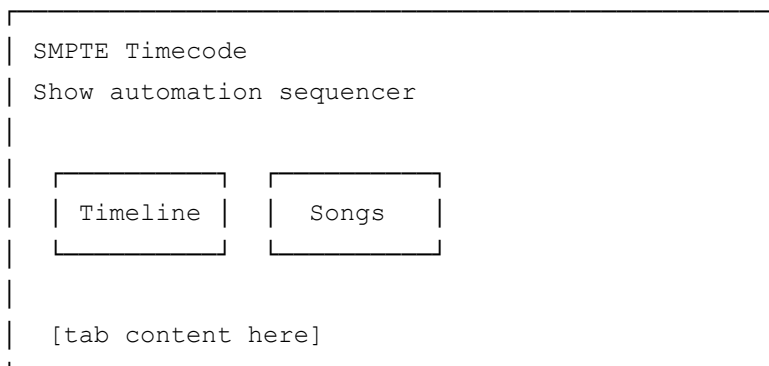
For `ProPresenter` mode: the operator selects which PP presentation/playlist corresponds to this song. The cue map then generates `pp-trigger-slide` events targeting the correct slide indices. This requires the Bridge to report available presentations (expand the PP polling to include `/v1/presentations` or `/v1/playlists`).

UI Design — Pages and Navigation

Navigation: new "Songs" tab in the sidebar

Add a "Songs" entry to the org sidebar navigation (between Rundown and Timecode, or as a sub-item under Timecode). Icon: `Music` from `lucide-react`.

Alternatively (and this may be the better UX): add a **"Songs" tab** on the existing Timecode page itself. The timecode page becomes a two-tab layout:

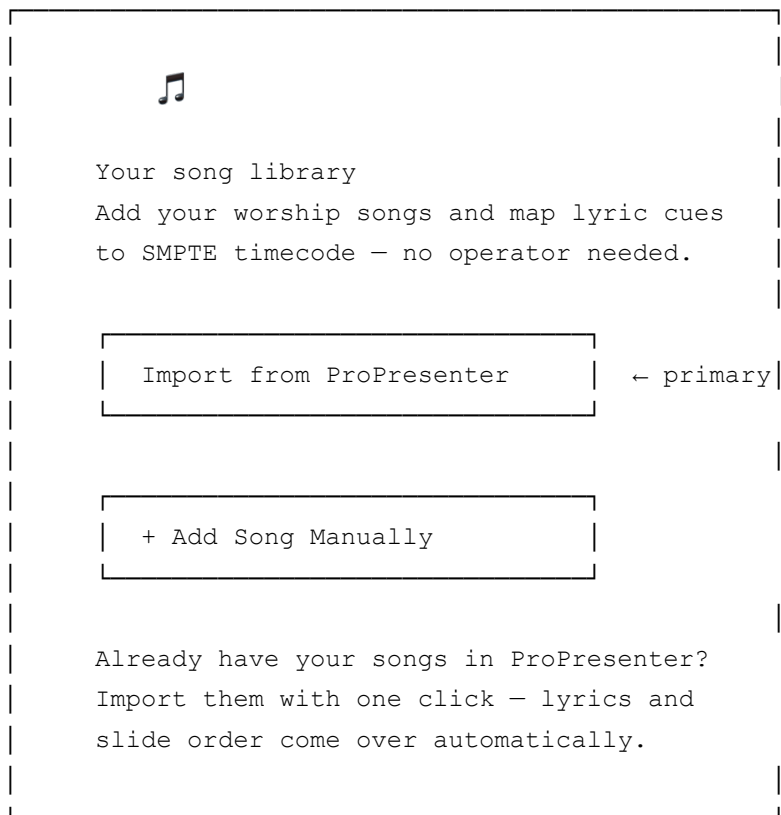


Recommendation: sidebar nav item. Songs are a first-class concept that teams will use independently of timecode (browsing their library, editing lyrics, building setlists). Burying them inside the timecode page limits discoverability. The timecode page gets a "Load Song Cues" button that deep-links to the song library with a "select and load" flow.

Page 1: Song Library (/\$slug/songs)

The entry point. Clean, inviting, minimal.

Empty state (critical — this is most users' first impression):



When Bridge is not connected or PP not configured, the "Import from ProPresenter" button shows in a disabled/muted state with a hint: "Connect the ShowPilot Bridge to your ProPresenter machine to import songs." The manual add button becomes primary.

Populated state:

- Search bar + "Add Song" button + "Import from PP" button top-right
- Card grid or compact list (user's choice, default grid)
- Each card: song title, artist, section count, cue map count, key/BPM if set
- PP-imported songs show a small ProPresenter badge/icon on the card

- Click → opens song detail/editor

Page 2: Song Editor (/\$slug/songs/\$songId)

Two-panel layout on desktop. Single-column scrollable on narrow screens.

Left panel: Sections

- Drag-to-reorder list of song sections
- Each section: label (editable inline), lyrics textarea
- "Add Section" button at bottom with quick-add presets: Verse, Chorus, Pre-Chorus, Bridge, Tag, Intro, Outro, Interlude
- Delete section (with confirmation if it has cues mapped)

Right panel: Cue Maps

- Dropdown to select/create cue map arrangement ("Default", "Youth Night", etc.)
- Visual timeline showing sections mapped to timecodes
- Two ways to create cues:

Method A — Manual entry: Table with columns: Timecode (HH:MM:SS:FF) | Section | Actions (delete) "Add Cue" row at bottom with timecode input + section dropdown

Method B — "Tap to Mark" (the killer UX): This is what makes the feature inviting. The operator:

1. Clicks "Tap to Mark" — enters marking mode
2. Plays the song in Ableton (or starts internal freerun for rehearsal)
3. ShowPilot shows the running timecode and the section list
4. The operator **taps each section** as the music reaches that point
5. ShowPilot records the current timecode as the cue point for that section
6. Done — the entire cue map is built in one listen-through

UI for Tap to Mark mode:

TAP TO MARK	[Done]
TC: 00:01:32:15	• Running (MTC)

Tap a section when you hear it start:

✓ Intro @ 00:00:00:00
✓ Verse 1 @ 00:00:12:05
✓ Chorus 1 @ 00:00:45:20
▶ Bridge [tap to mark] ← next
Chorus 2 [waiting]
Outro [waiting]

Tip: You can adjust timecodes after marking.

The next unmarked section is highlighted. Tapping it (click or keyboard shortcut — spacebar or Enter) captures the current TC. The list auto-advances to the next section. After all sections are marked, the mode auto-exits and the cue map is saved.

Keyboard shortcut: **Spacebar** to mark the next section (same muscle memory as tapping a click track). This is the single most important UX detail — it needs to feel like tapping along to the music.

Editing after marking: each cue's timecode is editable. Click the timecode to type a new value, or use +/- frame nudge buttons for fine-tuning.

Page 3: Live Lyrics Preview (on the Timecode page)

On the existing Timecode page, when song cues are loaded into the automation timeline, show a live lyrics preview panel:

• LIVE LYRICS PREVIEW

Amazing Grace
how sweet the sound
that saved a wretch like me

NEXT: Verse 2 — "I once was lost..."

TC: 00:01:32:15 Section: Verse 1

This gives the operator confidence that the automation is working without needing to check the kiosk screen.

Kiosk: Lyrics output route

Extend the existing kiosk/stage display system. When lyrics are active, the kiosk shows:

- Full-screen lyrics text (large, centered, high contrast)
- Smooth crossfade transitions between sections
- Song title as a subtle overlay (top or bottom, configurable)
- Optional: timecode overlay for confidence monitoring

The kiosk subscribes to the TimecodeRelay WebSocket. When a `lyrics-update` message arrives, it transitions to the new section.

Timecode Page Integration

On the existing timecode page, add to the automation event editor:

1. A new **"Load Song Cues"** button (prominent, next to "Load Demo Show")
2. When clicked, opens a modal/sheet showing the song library
3. Operator selects a song and a cue map arrangement
4. Selects target: "ShowPilot Native Lyrics" or "ProPresenter" (if Bridge is connected)
5. Cues are bulk-added to the automation timeline as `lyrics-goto` or `pp-trigger-slide` events
6. The lyrics preview panel appears

The "Load Song Cues" button should be the primary CTA on the timecode page when no events are loaded — more prominent than "Load Demo Show."

Output Targets — Configuration

In org settings (or a settings panel on the Songs page), the operator configures the lyrics output:

Target: (radio buttons)

- **ShowPilot Native** — renders on kiosk/stage display (default)
- **ProPresenter** — sends slide triggers to connected PP instance via Bridge

- **Both** — native display AND ProPresenter simultaneously

ProPresenter settings (when PP target selected):

- Presentation mapping: which PP presentation corresponds to each song
- Slide index mapping: auto-mapped by section order, or manually overridden
- Requires Bridge connected with PP configured

File Structure — New Files

```
apps/web/src/
├── types/
│   └── timecode.ts           # MODIFY: add new action types + lyrics WS
├── lib/
│   └── songs.ts             # NEW: song CRUD server functions
├── routes/
│   └── $slug/
│       ├── songs.tsx        # NEW: song library page
│       └── songs.$songId.tsx # NEW: song editor + cue map page
├── components/
│   ├── songs/
│   │   ├── SongLibrary.tsx  # NEW: song list/grid with search + empty
│   │   ├── SongCard.tsx     # NEW: song card for library grid
│   │   ├── SongEditor.tsx   # NEW: section editor (left panel)
│   │   ├── SongSectionEditor.tsx # NEW: individual section label + lyrics
│   │   ├── CueMapEditor.tsx  # NEW: cue map timeline + manual entry
│   │   ├── TapToMark.tsx     # NEW: tap-to-mark cue recording mode
│   │   ├── ImportFromPPModal.tsx # NEW: ProPresenter presentation picker +
│   │   └── LoadSongCuesModal.tsx # NEW: modal for loading cues into timecod
│   ├── lyrics/
│   │   ├── LyricsDisplay.tsx # NEW: full-screen lyrics renderer
│   │   ├── LyricsPreview.tsx # NEW: compact preview for timecode page
│   │   └── LyricsKioskView.tsx # NEW: kiosk route wrapper
│   └── timecode/
│       ├── AutomationEventEditor.tsx # MODIFY: add lyrics actions to dropdown
│       └── AutomationTimeline.tsx    # MODIFY: add lyrics action icons/colors
└── durable-objects/
```

```

|   └─ TimecodeRelay.ts           # MODIFY: add lyrics-goto + pp-trigger exec
|
└─ hooks/
|   └─ useSongs.ts               # NEW: song CRUD hook with optimistic UI
|   └─ usePPImport.ts           # NEW: hook for PP presentation query + im
|
└─ prisma/
    └─ migrations/
        └─ NNNN_songs.sql       # NEW: songs schema

apps/bridge/ (Tauri app - Rust backend + web frontend)
└─ src-tauri/src/
|   └─ midi/
|       └─ mod.rs               # NEW: MIDI module - enumerate inputs, dec
|       └─ mtc_decoder.rs       # NEW: MTC quarter-frame decoder → Timecod
|       └─ commands.rs         # NEW: Tauri commands: list_midi_inputs, s
|       └─ protocols/
|           └─ propresenter.rs  # MODIFY: add query-presentations, trigger
|
└─ src/ (Tauri frontend)
    └─ components/
        └─ MidiSettings.tsx     # NEW: MIDI input selector + MTC status in

```

Permissions

Add to `src/lib/permissions.ts`:

```

"songs:access"    // View song library and lyrics
"songs:manage"    // Create, edit, delete songs, sections, cue maps

```

Add both to `ADMIN_TIER_PERMISSIONS` (td, cd, pd get full access). Add `songs:access` to the `member` role default.

Implementation Order

Phase 1 — Data model + Song CRUD + ProPresenter import (ship first, no timecode dependency)

1. Migration: `songs`, `song_sections`, `song_cue_maps`, `song_cues` tables
2. Prisma schema update + `db:generate`

3. Server functions in `songs.ts`
4. Song library page with empty state (PP import primary CTA, manual add secondary)
5. Song editor with section management (drag-to-reorder, inline edit)
6. Bridge: add `query-presentations` command — queries PP's `/v1/presentations` and `/v1/presentation/{uuid}` APIs, returns full slide content
7. `ImportFromPPModal` — presentation picker, bulk import, section auto-creation from PP slides
8. `usePPImport` hook for the query/import flow via BridgeRelay

Why first: Teams can populate their song library on day one by importing from ProPresenter — zero manual lyric entry for churches that already use PP. This is the fastest path to "wow, this actually works." Also useful without timecode: it's a lyrics database with a better import story than anything else in the market.

Phase 2 — Cue mapping + Tap to Mark

1. Cue map CRUD (arrangements per song)
2. Manual cue entry (timecode + section picker)
3. Tap to Mark mode (spacebar-driven cue recording)
4. Cue map visual timeline

Phase 3 — Native lyrics engine

1. `lyrics-goto` and `lyrics-clear` automation action types
2. TimecodeRelay execution handler + `lyrics-update` WS broadcast
3. LyricsDisplay renderer component
4. Kiosk route integration (lyrics output on stage display)
5. "Load Song Cues" flow on timecode page
6. Live lyrics preview panel on timecode page

Phase 4 — ProPresenter slide triggering

1. Bridge: expand `sendCommand` to accept payload object
2. Bridge: add `trigger-slide` and `trigger-playlist-item` PP API endpoints

3. `pp-trigger-slide` / `pp-trigger-next` / `pp-trigger-clear` automation action types
4. PP slide mapping UI in Load Song Cues modal (auto-mapped for PP-imported songs, manual for others)
5. BridgeRelay command forwarding from TimecodeRelay to Bridge
6. For PP-imported songs, cue map auto-generates `pp-trigger-slide` events with correct indices — no mapping step needed

Phase 5 — Bridge MTC input (Tauri / Rust)

1. `midir` crate integration — enumerate MIDI inputs, open selected port
2. MTC quarter-frame decoder (accumulate 8 quarter-frames → full TimecodeValue)
3. Frame rate auto-detection from MTC type bits
4. Feed decoded timecode to Bridge's WebSocket as `feed-tc` at 10Hz
5. MIDI input selector UI in Bridge Tauri app
6. Hot-plug handling: re-enumerate on device connect/disconnect, auto-reconnect
7. Ableton setup documentation (user-facing)

Why last: The browser-based Web MIDI path and the internal freerun generator already work for development and testing of Phases 1–4. The Bridge MTC input is the production-grade path and can be built in parallel without blocking the lyrics feature. Ship it when it's solid — this is the path that has to survive a live Sunday morning.

Edge Cases and Gotchas

1. **Timecode rewind/loop:** When Ableton loops a section (common in rehearsal), `crossedTriggerFrame` will re-fire events. The existing `fired` flag prevents double-fires, but a `reset-events` must happen on rewind. The `evaluateEvents` function already handles this via the `previousFrame > currentFrame` detection — verify it resets `fired` flags on rewind. If not, add that logic.
2. **Multiple songs in a set:** Each song in a worship set has its own timecode range. Either: (a) each song starts at 00:00:00:00 and the operator loads cues per song, resetting between songs; or (b) songs are stacked with offsets (Song 1 at 00:00, Song 2 at 00:05:30, etc.) for a continuous set. Support both. The "Load Song Cues" modal should have an "offset" field for stacking.

3. **Tolerance frames:** Lyrics cues should default to `toleranceFrames: 5` (more forgiving than device triggers) because the human perception of "lyrics appeared on time" has wider tolerance than a lighting cue.
4. **Rehearsal without Ableton:** The internal freerun generator lets operators test cue maps without a DAW connected. The Tap to Mark mode should work with both MTC and internal freerun.
5. **Section repeats:** A song often repeats the Chorus. The cue map must allow the same `sectionId` to appear at multiple timecodes. The `song_cues` table supports this naturally.
6. **No lyrics operator needed — but one should be able to override:** If an operator is watching and wants to manually advance (band went off-script), they should be able to click a section in the live preview to jump to it, overriding the automation temporarily.
7. **ProPresenter slide count mismatch:** For manually-created songs targeting PP output, the mapping UI must surface any mismatch clearly (8 ShowPilot sections vs 10 PP slides) and let the operator manually map. For PP-imported songs, this mismatch shouldn't happen initially — but if the operator edits sections after import, or the PP presentation is updated, the re-import/sync flow in the import modal handles reconciliation.
8. **CCLI compliance note:** ShowPilot stores lyrics the user enters or imports from ProPresenter. It does not (yet) integrate with SongSelect API. Add a CCLI number field so churches can track licensing. Future: SongSelect API integration for auto-import.
9. **ProPresenter not running during import:** The Bridge queries PP's HTTP API. If PP is closed or unreachable, the import flow shows a clear error: "Can't reach ProPresenter at 192.168.1.50:1025. Make sure ProPresenter is running and the Network port is enabled in Preferences → Network." Don't fail silently.
10. **PP presentation changes after import:** If the worship leader edits lyrics in PP during the week, the ShowPilot copy is stale. The re-import flow detects changes by comparing slide text. Surface this proactively: when the operator opens a PP-imported song, a "Check for updates" button queries the Bridge for the latest version. If changes are found, show a diff and let the operator merge or replace.

Non-Goals (Explicitly Out of Scope)

- SongSelect / Planning Center API integration (future — ProPresenter import + manual paste for now)

- Lyric formatting (bold, italic, chord charts) — plain text only for V1
 - Multi-language lyrics — future
 - Video backgrounds behind lyrics — future
 - Ableton Session Link / OSC from Ableton — MTC is sufficient and already works
 - Building an Ableton plugin — unnecessary, MTC is the standard protocol
 - MIDI note triggers (using MIDI notes instead of timecode for cues) — future consideration but timecode is the right V1
 - EasyWorship / MediaShout import — future, same pattern as PP import but lower priority
 - Two-way PP sync (pushing lyrics changes back to ProPresenter) — one-way import only for V1
-

Success Criteria

1. A worship team can import their entire song library from ProPresenter with one click, use Tap to Mark to build cue maps in one listen-through per song, and have lyrics auto-fire during a live service
2. Teams using ProPresenter keep using it — ShowPilot drives slide transitions via timecode instead of a human operator, with zero manual slide-index mapping for PP-imported songs
3. Teams without ProPresenter get a native lyrics engine on ShowPilot's kiosk/stage display that's good enough to replace PP for lyrics-only use cases
4. The empty state leads with ProPresenter import — the fastest path from "I just signed up" to "my songs are in here"
5. The Tap to Mark experience feels musical — spacebar tap, auto-advance, done in one listen-through
6. A church with no Ableton can still use internal freerun to manually test and run cue maps (the feature degrades gracefully without a DAW)
7. The Ableton → Bridge MTC → timecode path works reliably on a live Sunday morning with no browser permission prompts, no Chrome dependency, no flakiness